

Project 8: Smart Traffic Light Control System

Team: Safiya Arif and Sehareena Saleem

Car:

Represents a vehicle with a unique ID

Lane:

Holds cars using a **queue**

- State (RED / GREEN)
- Waiting Time
- Queue length

Intersection:

Contains **4 lanes**: North, South, East, West

Dequeues and Enqueues cars based on the greenTime.

Adaptive Controller

Controls the whole simulation

Generates cars

Allocates green lights

Moves cars between intersections

What does our system do?

Key Features:

1. **Dynamic Traffic Generation** – Cars generated randomly in source lanes.
2. **Adaptive Green Lights** – Lanes with longer queues or higher waiting time get priority.
3. **Lane Queues** – Cars move in FIFO order from one intersection to the next.
4. **Outgoing Connections** – Each lane can send cars to a specific lane in the next intersection.
5. **Cars Leave Network** – Once a car reaches the end of its path, it exits the network.
6. **Real-time Updates** – System tracks number of cars, waiting time, and lane states (RED/GREEN).

Data Structures Used:

- **Queues** → lanes (FIFO car movement)
- **Priority Queue** → adaptive green-light selection
- **Hash Map** → fast lookup of outgoing lane connections

How did we do it?

1. Implemented our own queue data structure via nodes and made all the necessary functions like enqueue, dequeue and others.
2. We made use of templates so that we can use it with data types of our desire.
3. We also made a basic lane class, it has a queue of vehicles.
4. Each queue(lane) is filled with random amount of vehicles (under max that is right now 10), so that north lane has 3 vehicles south has 9 etc. That is the idea of setting up a scenario for intersection.
5. Priority queue assigns the lane for greenTime on the basis of waiting time and no Of cars (queue length basically).
6. We have implemented intersection each having (N S E W).

```

13 class Lane{
14     int laneId;
15     std::string name;//N-S E-W
16
17     int greenTime; //assigned by algo
18     NodeBasedQueue <Car*> cars;
19
20 public:
21     int waitingTime;
22     std::string state; //red or green
23     bool isSourceLane;
24     NodeBasedQueue<Car*> lane; //will fill vehicles in it
25
26     Lane(int id, std::string n);
27     void addCar(Car* car);
28     Car* removeCar();
29     Car* peekCar() const;
30
31     int getLaneLength() const;
32     bool isEmpty() const;
33     std::string getName() const;
34     int getId() const;
35     int getGreenTime() const;
36     void setGreenTime(int g);
37     void setState(std::string s);
38     std::string getState();
39     void resetWaitingTime();
40     void incrementWaitingTime();
41     void updateGreenTime(int greenTime);
42     void display() const; //display no of cars by a * in that particular lane
43
44 };

```

```

class Intersection{
public:
    int intersectionId;

    std::unordered_map<std::string, Lane*> Lanes;
    Intersection();
    ~Intersection();
    void processGreen(const std::string& lane1,
                     const std::string& lane2,
                     int greenTime);
    void updateWaitingTimes(const std::string& g1, const std::string& g2);
    Lane* getLane(const std::string& name);
    std::unordered_map<std::string, Lane*>& getAllLanes();
    bool addCarToLane(const std::string &name, Car* car);
    std::unordered_map<std::string, std::pair<Intersection*, std::string>> outgoing;
};

#endif

```

Our current repo structure:

Files

main

Go to file

docs

Project Charter.pdf

Smart-Traffic-Light-Controller-w...

trafflights.jpg

src

.gitignore

controller.cpp

controller.h

main.cpp

queue.h

structure.cpp

structure.h

DSA_Project / src

starrynightcoded Create controller.h

Name	Last commit message
..	
.gitignore	uploading queue in
controller.cpp	Create controller.c
controller.h	Create controller.h
main.cpp	Update main.cpp
queue.h	Update queue.h
structure.cpp	Create structure.cp
structure.h	Create structure.h

Explanation of the structure:

1. The docs folder has all the design docs and documentation.
2. The src folder will have all the modules.
3. The structure.cpp/.h has all the necessary classes like Lane, intersection and Car (Vehicle basically).
4. The controller handles the simulation/stimulation and the core adaptive property.
5. main() creates intersections and connects them then calls the controller to run it for us.

ScreenShot of example run:

```
Traffic Network Diagram:
W1          W2
|           |
N1 ---- A --- S1 ----- N2 ----- B ----- S2
|           |
E1          E2
|           |
W3          |
|           |
N3 ---- C ---- S3
|           |
E3          |

Starting Simulation...
```

=== Simulation Step 1 ===

Generated Car 1 in lane E of Intersection A
Generated Car 2 in lane E of Intersection A
Generated Car 3 in lane N of Intersection A
Generated Car 4 in lane N of Intersection A
Generated Car 5 in lane N of Intersection A
Generated Car 6 in lane S of Intersection B
Generated Car 7 in lane S of Intersection B
Generated Car 8 in lane S of Intersection B
Generated Car 9 in lane W of Intersection C
Generated Car 10 in lane W of Intersection C
Generated Car 11 in lane W of Intersection C

Intersection A: Allocating GREEN to N & S for 3 time units

Intersection B: Allocating GREEN to N & S for 2 time units

Intersection C: Allocating GREEN to E & W for 2 time units

Car 8 moved from lane S of Intersection B to lane S of next Intersection

Car 11 moved from lane W of Intersection C to lane E of next Intersection

What are our next steps:

1. Implement logs + Performance Metrics Calculation
2. Make better readable visualizations. (on the console).
3. Improve algorithm + do unit testings.